

Lab 1: Reproducible Training Pipeline & Production Containerization

Complete Implementation Blueprint, Engineering Walkthrough, and Verification Report

STUDENT NAME & ID Peerawat Somsurat (ID: 6688218)	OFFICIAL SUBMISSION REPOSITORY https://github.com/PeerawatSon/itcs355-lab1.git
CLOUD TARGET Microsoft Azure (Blob Storage + Azure Container Registry)	VERIFICATION RESULT 13 / 13 MECHANICAL CHECKS PASSED (100%)

1. Executive Summary & Core Objectives

The central thesis of modern Machine Learning Operations (MLOps) is that *"a machine learning model is code, data, and environment unified into a single repeatable artefact."* In traditional exploratory workflows, models suffer from **silent environmental drift**: non-pinned dependencies pick up subtle wheel republishing differences, container base images change unpredictably underneath tags, and random shuffling leaks persistent entity properties across data splits.

ITCS355 Lab 1 mandates transforming an ad-hoc ML model into an irrefutably reproducible pipeline. The primary criterion is straightforward: *a stranger on a different operating system and CPU architecture must clone the repository, run exactly one command (`make reproduce`), and verify that the resulting ROC-AUC matches the author's claimed value within honest empirical tolerance—without requiring any private credentials or communication.*

The Reproducibility Quad: Four Interlocking Links

Algorithmic reproducibility succeeds only when four independent links remain unbroken:

- Versioned Data:** Byte-level integrity tracked by DVC (SHA/MD5 hashes), never relying on floating endpoints.
- Hashed Dependencies:** Cryptographic SHA-256 package hashes preventing supply-chain wheel mutations.
- Digest-Pinned Image:** Container base images pinned to immutable `@sha256:...` digests, defeating tag mutation.
- Controlled Seeds:** Deterministic pseudorandom state across Python, NumPy, and Scikit-Learn.

2. Problem Formulation & Data Contract

2.1 Industrial Sensor Telemetry & Machine Failure Prediction

The workload simulates an industrial maintenance setting predicting component failure within 7 days from high-frequency multi-sensor streams. The raw telemetry consists of:

- **240 physical machines**, each recording 25 temporal sensor observations (total **6,000 observations**).
- **6 continuous sensor features**: `temp_c`, `vibration_mms`, `pressure_kpa`, `rpm`, `voltage_v`, and `current_a`.
- **Binary Target**: `failed_within_7d`, exhibiting severe real-world class imbalance (~**11.7%** positive failure rate).

2.2 Group Leakage Prevention: GroupKFold by machine_id

Physical machines possess persistent individual baseline characteristics (e.g., a motor that naturally runs 4°C hotter or exhibits baseline harmonic vibration). If data is partitioned row-wise using standard uniform random sampling (`train_test_split`), rows from Machine #42 appear in both the training set and the test set. The decision forest will simply *memorize machine identifiers* via its physical baseline rather than discovering genuine wear degradation patterns.

The Critical Danger of Group Leakage

Row-wise splitting yields artificially inflated validation metrics (e.g., ROC-AUC > 0.94) that completely collapse in production when evaluating previously unseen machinery. Lab 1 enforces **Group-Based Partitioning** grouped by `machine_id` (60% Train, 20% Validation, 20% Test). An automated unit test (`test_no_machine_leaks_across_splits`) enforces zero partition overlap.

3. System Architecture: The 3-Layer Design Pattern

To guarantee complete cloud portability without vendor lock-in, the codebase strictly decouples business logic from cloud providers using a three-layer boundary:

Architectural Layer	Component Scope	Design Contract & Guardrails
Layer 1: Provider-Neutral Logic	<code>src/</code> , <code>service/</code> , <code>monitoring/</code> , <code>tests/</code>	Pure Python & Scikit-Learn. Strictly forbidden from importing cloud SDKs (<code>boto3</code> , <code>google.cloud</code> , <code>azure</code>) or hardcoding bucket names/endpoints.
Layer 2: Environment Contract	<code>cloud.env</code> , <code>src/config.py</code>	Resolves eight standardized capability slots (<code>CLOUD_PROVIDER</code> , <code>BLOB_URI</code> , <code>CONTAINER_REGISTRY</code> , etc.).
Layer 3: Cloud Seam (Adapter)	<code>cloudlayer/</code> (<code>base.py</code> , <code>azure.py</code> , <code>aws.py</code> , <code>gcp.py</code>)	The <i>only</i> layer where provider SDKs are imported. Exposes uniform methods: <code>upload()</code> , <code>download()</code> , <code>push_image()</code> .

4. Comprehensive Task-by-Task Implementation Walkthrough

Task 1: Production Scaffolding & Portability Seam

Layout & Hygiene

WHAT TO DO

Refactor messy Jupyter notebooks into a production repository structure. Establish single-point environment resolution via `src/config.py`, set up a deterministic data generation pipeline, and establish static analysis guardrails preventing cloud SDK leakage into core code.

HOW TO DO IT (STEP-BY-STEP)

1. Establish Canonical Directory Hierarchy:

```
itcs355-lab1/
├─ Dockerfile           # Multi-stage container definition
├─ Makefile             # Grader commands (reproduce, verify, test)
├─ requirements.in      # Unpinned direct dependency declarations
├─ requirements.txt      # Cryptographically hashed lockfile
├─ cloud.env.example    # Template of 8 capability slots
├─ cloudlayer/azure.py  # Isolated Azure SDK implementation
├─ data/raw.dvc         # DVC data version pointer
├─ src/                 # Provider-neutral training and data modules
├─ reports/mlflow.db    # SQLite local experiment tracking store
└─ tests/test_data.py   # Automated data contract & leakage tests
```

2. **Implement Environment Contract (`src/config.py`):** Implement `AppConfig` dataclass parsing `cloud.env` with rigorous validation. Paths must resolve relative to repository roots via `Path(__file__).resolve()`, never using hardcoded machine paths.

3. **Execute Portability Audit:** Run static AST search to assert zero provider strings exist in Layer 1:

```
make portability-audit
# Output: PORTABILITY AUDIT PASSED - src, service, monitoring, tests contain no provider-s
```

Task 2: Cryptographic Environment & Digest Pinning

Reproducibility Core

WHAT TO DO

Standard version pinning (e.g. `scikit-learn==1.8.0`) is vulnerable to supply-chain attacks and republished wheels. Generate an exhaustive lockfile containing SHA-256 hashes for all wheels and source distributions. Pin the container base image to its exact 64-character SHA-256 manifest digest. Control random seeds across all libraries. Formulate a technical defense regarding reproducibility trade-offs.

HOW TO DO IT (STEP-BY-STEP)

1. **Compile Cryptographic Package Hashes:** Use `uv` or `pip-compile` targeting Linux x86_64 and Python 3.11:

```
uv pip compile --python-platform x86_64-unknown-linux-gnu --python-version
```

Verify that every package block contains `--hash=sha256:...` directives.

2. **Retrieve Base Image Digest & Pin Dockerfile:**

```
docker pull python:3.11-slim
docker inspect --format='{{index .RepoDigests 0}}' python:3.11-slim
# Output: python@sha256:9534e5a8e315485d4061ed659af0fd78a284c015f9b73661b41d6bab25604534
```

Pin both stages in `Dockerfile` and enforce `pip install --require-hashes`.

3. **Implement Deterministic Pseudorandom Seeding (`src/seeds.py`):**

```
import random, os, numpy as np
def set_seeds(seed: int = 20260101):
    random.seed(seed)
    np.random.seed(seed)
    os.environ["PYTHONHASHSEED"] = str(seed)
```

Graded Trade-off Defense (as committed in README.md)

*"Under severe time pressure, I would drop **seed control** first. Dropping seeds preserves container buildability and dependency integrity—the container still builds and executes cleanly without breaking changes. What breaks is exact numerical parity and direct comparability between individual training runs; test metrics fluctuate slightly (within $\sim \pm 0.02$) due to stochastic data splitting and tree bootstrapping. However, an executable pipeline with minor metric variance is preferable under tight deadlines to a failed build or an unverified dependency chain."*

Task 3: Production Multi-Stage Containerization

Container Engineering

WHAT TO DO

Build an immutable Docker container capable of executing training end-to-end. Implement a multi-stage architecture to eliminate build toolchains from the runtime image, run as an unprivileged non-root user for security compliance, enforce `linux/amd64` cross-compilation (crucial for Apple Silicon developers), and implement cloud registry push capabilities.

HOW TO DO IT (STEP-BY-STEP)

1. Author the Multi-Stage Dockerfile:

```
# STAGE 1: Builder (compiler and dependency installation)
FROM python:3.11-slim@sha256:9534e5a8e315485d4061ed659af0fd78a284c015f9b73661b41d6bab25604
WORKDIR /build
COPY requirements.txt ./
RUN pip install --require-hashes --prefix=/install -r requirements.txt

# STAGE 2: Runtime (minimal footprint, non-root user)
FROM python:3.11-slim@sha256:9534e5a8e315485d4061ed659af0fd78a284c015f9b73661b41d6bab25604
RUN useradd --create-home --uid 10001 runner
ENV PYTHONUNBUFFERED=1 PYTHONDONTWRITEBYTECODE=1 PYTHONPATH=/app
COPY --from=builder /install /usr/local
RUN mkdir -p /app && chown runner:runner /app
WORKDIR /app
COPY --chown=runner:runner src/ ./src/
COPY --chown=runner:runner cloudlayer/ ./cloudlayer/
COPY --chown=runner:runner scripts/ ./scripts/
USER runner
ENTRYPOINT ["python", "-m", "src.train"]
CMD ["--n-estimators", "200", "--max-depth", "8"]
```

2. Compile for linux/amd64 Architecture:

```
docker buildx build --platform linux/amd64 -t itcs355-lab1:latest --load .
```

3. Implement Azure Container Registry Push (`cloudlayer/azure.py`): Automate authentication via `az acr login`, tag image for remote ACR, push, and parse the immutable SHA-256 digest:

```
# push_image returns: itcs3556688218.azurecr.io/itcs355@sha256:ada790b688df98...
```

Task 4: Data Version Control & Anti-Leakage Testing

Data Lineage

WHAT TO DO

Decouple large raw datasets from Git history using Data Version Control (DVC). Configure remote object storage backed by Azure Blob Storage. Implement deterministic grouped train/val/test data splitting in Python, and author an automated test suite verifying zero data leakage across partitions.

HOW TO DO IT (STEP-BY-STEP)

1. Initialize & Configure DVC with Azure Blob Storage:

```
dvc init
dvc remote add -d storage azure://itcs355/itcs355/dvc
dvc config core.remote storage
dvc config remote.storage.account_name itcs3556688218
dvc add data/raw
git add data/raw.dvc .dvc/config .dvcignore
dvc push
```

2. Implement Deterministic Group Partitioning (src/data.py):

```
def split(df: pd.DataFrame, seed: int = 20260101):
    machines = np.array(sorted(df["machine_id"].unique()))
    rng = np.random.default_rng(seed)
    rng.shuffle(machines)
    n = len(machines)
    train_m = set(machines[: int(0.6 * n)])
    val_m = set(machines[int(0.6 * n) : int(0.8 * n)])
    test_m = set(machines[int(0.8 * n) :])
    return df[df["machine_id"].isin(train_m)], df[df["machine_id"].isin(val_m)], df[df["ma
```

3. Implement Group Leakage Assertion (tests/test_data.py):

```
def test_no_machine_leaks_across_splits():
    df = data.load_raw()
    train_df, val_df, test_df = data.split(df, seed=20260101)
    s_train, s_val, s_test = set(train_df["machine_id"]), set(val_df["machine_id"]), set(t
    assert s_train.isdisjoint(s_val), "Train and Val share machines!"
    assert s_train.isdisjoint(s_test), "Train and Test share machines!"
    assert s_val.isdisjoint(s_test), "Val and Test share machines!"
```

Task 5: Hyperparameter Study & MLflow Tracking

Experimentation

WHAT TO DO

Conduct a controlled hyperparameter study consisting of at least five runs with deliberate architectural variations (not simply seeds). Log all parameters, separately track Validation and Test performance (ROC-AUC and PR-AUC), attach complete data lineage tags (DVC hash, git commit, data fingerprint), and persist trained model artifacts.

HOW TO DO IT (STEP-BY-STEP)

1. **Configure Local Tracking Store:** Point MLflow to local SQLite file:

`MLFLOW_TRACKING_URI=sqlite:///reports/mlflow.db`.

2. **Execute Systematic Study (5 Variations):** Execute runs varying tree depth (`max_depth` $\in \{4, 6, 8, 12\}$), minimum leaf size (`min_samples_leaf` $\in \{1, 5\}$), and ensemble size (`n_estimators` $\in \{100, 200\}$).

EMPIRICAL STUDY RESULTS MATRIX (RETRIEVED FROM SQLITE)

Run Name	Trees	Depth	Min Leaf	Val ROC	Val PR	Test ROC	Test PR	Artifact Logged
shallow-d4-n100	100	4	5	0.8426	0.3932	0.8533	0.4965	model.skops (v1)
medium-d6-n100	100	6	5	0.8434	0.4019	0.8493	0.4943	model.skops (v2)
angry-ape-304 (Baseline)	200	8	5	0.8364	0.3963	0.8482	0.4776	model.skops (v3)
low-reg-leaf1	200	8	1	0.8318	0.3861	0.8483	0.4725	model.skops (v4)
deep-d12-n200	200	12	5	0.8352	0.3773	0.8424	0.4680	model.skops (v5)

Task 6: The Graded README, Tolerance & One Command

Verification & Submission

WHAT TO DO

Write a self-contained README enabling single-command verification. Specify the claimed test ROC-AUC with an empirical tolerance backed by study variance. Resolve any container filesystem permission anomalies occurring during fresh clones.

HOW TO DO IT (STEP-BY-STEP)

1. **Establish the Claim Line & Tolerance:** Across the five intentional hyperparameter runs, test ROC-AUC varies between `0.8424` and `0.8533` (mean ≈ 0.848 , empirical spread $\approx \pm 0.005$). Establishing ± 0.010 provides a rigorous, honest statistical tolerance that passes evaluation without concealing noise:

```
expected test_roc_auc: 0.848 ± 0.010
```

2. **Critical Fix: The Fresh-Clone `reports/` Permission Trap:** *Problem:* Git does not track empty directories. When a grader clones a fresh repository and executes `make reproduce`, Docker automatically creates the volume mount target `reports/` as `root:root` (`0755`). The unprivileged in-container user `runner` (UID `10001`) is denied permission to create `mlflow.db`, crashing with `sqlite3.OperationalError: unable to open database file`.

Solution: Two-fold mitigation applied:

- Added `reports/.gitkeep` to track the directory in Git with normal user permissions.
- Prepended `mkdir -p reports` to the `reproduce:` recipe in `Makefile` before invoking `docker run`.

5. Official Test Suite Verification Results (`grade_lab1.sh`)

The repository was evaluated by executing the official course grading script against a fresh temporary clone:

Category	Check Item	Result	Details
1. Hygiene	cloud.env not committed	PASS	Zero credential leaks in Git log
	no credentials in history	PASS	Regex audit clean (no API/RSA keys)
	README has a claim line	PASS	Found expected test_roc_auc: ...
	no REPLACE blocks remain	PASS	All instructions replaced
2. Reproducibility	dependencies carry hashes	PASS	--hash=sha256: in requirements.txt
	base image pinned by digest	PASS	python:3.11-slim@sha256:...
	Dockerfile uses non-root	PASS	USER runner (UID 10001)
	no credentials baked in image	PASS	Zero ARG/ENV secret declarations
3. Quality Gates	portability audit clean	PASS	Zero cloud SDK leaks in Layer 1
	data tests pass	PASS	26 unit tests passed
	leakage test present	PASS	test_no_machine_leaks_across_splits
4. The One Command	make reproduce succeeds	PASS	Clean end-to-end container run
	metric matches the claim	PASS	Actual: 0.8482 (Delta: +0.0002)

6. Oral Defense & Drill 1 Preparation Cheat-Sheet

In Session 2, the instructor conducts **Drill 1**, evaluating individual student comprehension through evidence questions extracted directly from the repository. Below are the authoritative answers for defense:

Evidence Question 1: What is your unique Data Fingerprint and DVC Hash?

Data Fingerprint (First 16 chars of SHA-256): 422cccb9136e8140

DVC Hash of Raw Data (data/raw.dvc): 1c886b512c8a5c9bf723da1cd119fc80.dir

Significance: Proves that the raw dataset generated by seed 20260101 has identical byte-level hash across all training runs.

Evidence Question 2: What parameters did you vary across your study, and what happened?

Across five distinct MLflow runs, I varied tree depth (`max_depth`: 4, 6, 8, 12), leaf regularization (`min_samples_leaf`: 1, 5), and ensemble size (`n_estimators`: 100, 200).

Observed Dynamics: Moderate tree depths (`max_depth=4` and `6`) achieved superior out-of-sample generalization (Test ROC-AUC: 0.8533 and 0.8493). Deep unconstrained trees (`max_depth=12`, `min_samples_leaf=1`) overfit sensor noise on individual training machines, causing test performance to degrade to 0.8424.

Evidence Question 3: How did you select and defend your tolerance (± 0.010)?

Defense: The five study configurations exhibited test ROC-AUC scores spanning 0.8424 to 0.8533 around the baseline of 0.8482. The empirical spread across model architectures is approximately 0.010. Specifying a tolerance tighter than ± 0.005 would be brittle to minor cross-architecture floating point variations, while a broad tolerance (± 0.05 or ± 0.10) reflects concealed non-determinism. A tolerance of ± 0.010 is statistically sound and honest.

Conceptual Question 4: Why drop Seed Control first under time pressure?

Defense: Three pillars guarantee reproducible builds: hashed requirements, digest-pinned base images, and seed control.

- Dropping **package hashes** exposes the build to upstream wheel republishing and dependency supply-chain poisoning.
- Dropping **base image digests** causes silent upstream build failures when base tags move (e.g. Debian security patches updating Python C-toolchains).
- Dropping **seed control** sacrifices exact numerical parity between runs, but *preserves 100% buildability and runtime execution*. Under extreme project deadlines, a container that compiles and executes with ± 0.01 metric variation is vastly superior to a container that fails to build.

7. Conclusion & Repository Submission Status

Lab 1 successfully implements every requirement of reproducible machine learning operations. All code, multi-stage container configurations, hashed lockfiles, and DVC pointers have been validated, committed, and pushed to the official remote:

- **Remote GitHub Repository:** <https://github.com/PeerawatSon/itcs355-lab1.git>
- **Verified HEAD Commit:** 8ca4745 (fix: ensure reports directory exists for non-root container during make reproduce)
- **Cloud Assets:** DVC storage configured on Azure Blob Storage; Docker image pushed to Azure Container Registry.